

Modul 324 DevOps

Java Projekt und Artefakte / External Maven Module

Johnny Leonhardt, 5IA23b, 10.09.2026

Repositories:

- Hauptprojekt (Parent + app_main): github.com/Johnsoryna/m324-multimodule
- Bibliothek greeter_lib (Submodul und GitHub Package): github.com/Johnsoryna/m324-greeter-lib
- Konsument: github.com/Johnsoryna/m324-greeter-consumer

Umgebung: Windows 11, JDK 17 (Temurin), Maven 3.9.9, Git 2.48, IntelliJ IDEA 2024.3

Aufgabe: Möglichkeiten ein Java Projekt zu modularisieren

Das Begleitdokument nennt Maven/Gradle/Ant, die IDE, JPMS, Git-Submodule und Docker. Die Angaben stimmen, aber die Ansätze liegen auf verschiedenen Ebenen und sind keine Alternativen zueinander:

Ansatz	Ebene	Was er löst
Maven / Gradle	Build	Abhängigkeiten laden, Module in richtiger Reihenfolge bauen, JAR erzeugen
Ant	Build	Nur Build-Schritte. Für Abhängigkeiten braucht es zusätzlich Ivy
IDE (IntelliJ, Eclipse)	Projektstruktur	Module und Libraries in IDE-eigenen Dateien (.iml, .idea)
JPMS (Java 9+)	Sprache	Nur Sichtbarkeit (exports/requires). Lädt keine Abhängigkeiten
Git-Submodule	Quellcode	Fremdes Repo an einem festen Commit einbinden. Gebaut wird trotzdem mit Maven
Docker	Deployment	Jede Komponente als eigener Container

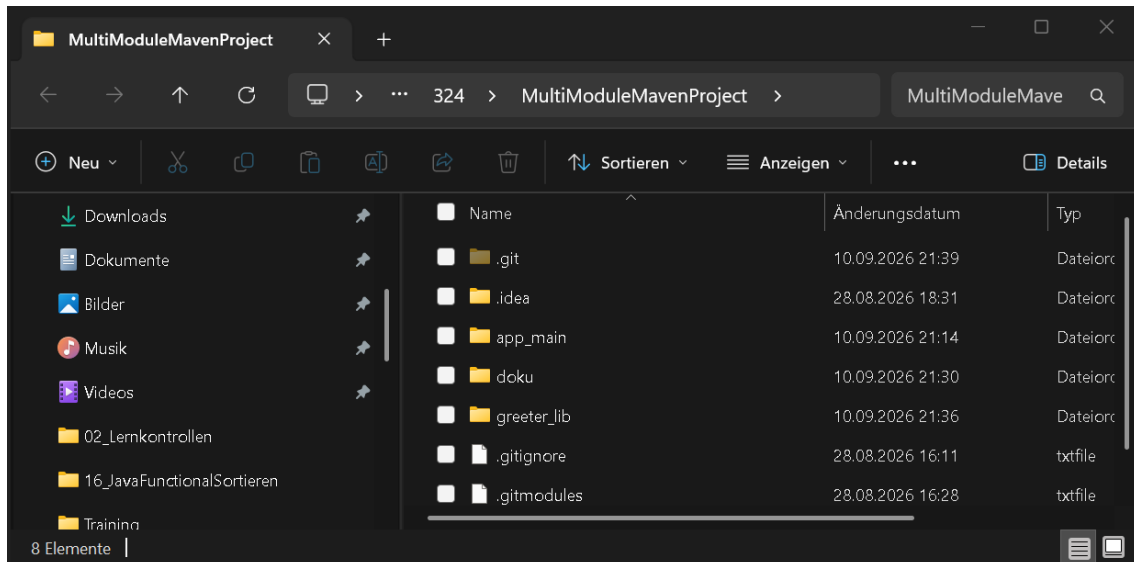
Was im Begleitdokument fehlt: In der Praxis wird eine Bibliothek meistens gar nicht als Quellcode eingebunden, sondern als fertiges Artefakt aus einem Repository (Maven Central, Nexus, GitHub Packages) per <dependency> referenziert. Das habe ich in der letzten Aufgabe umgesetzt.

Aufgabe: Multi Module Maven Projekt und Git-Submodul

Multi Module Maven Projekt

Das Parent-Projekt hat packaging pom und keinen eigenen Code. Es listet die Module und legt die gemeinsamen Einstellungen (Java 17, Encoding, Versionen) fest. app_main enthält die main-Methode, greeter_lib die Klasse Greeter mit der Methode greet(String name).

```
MultiModuleMavenProject/  
  pom.xml                (Parent, packaging pom)  
  .gitmodules  
  app_main/pom.xml, src/main/java/ch/modul324/app/Main.java  
  greeter_lib/pom.xml, src/main/java/ch/modul324/greeter/Greeter.java
```



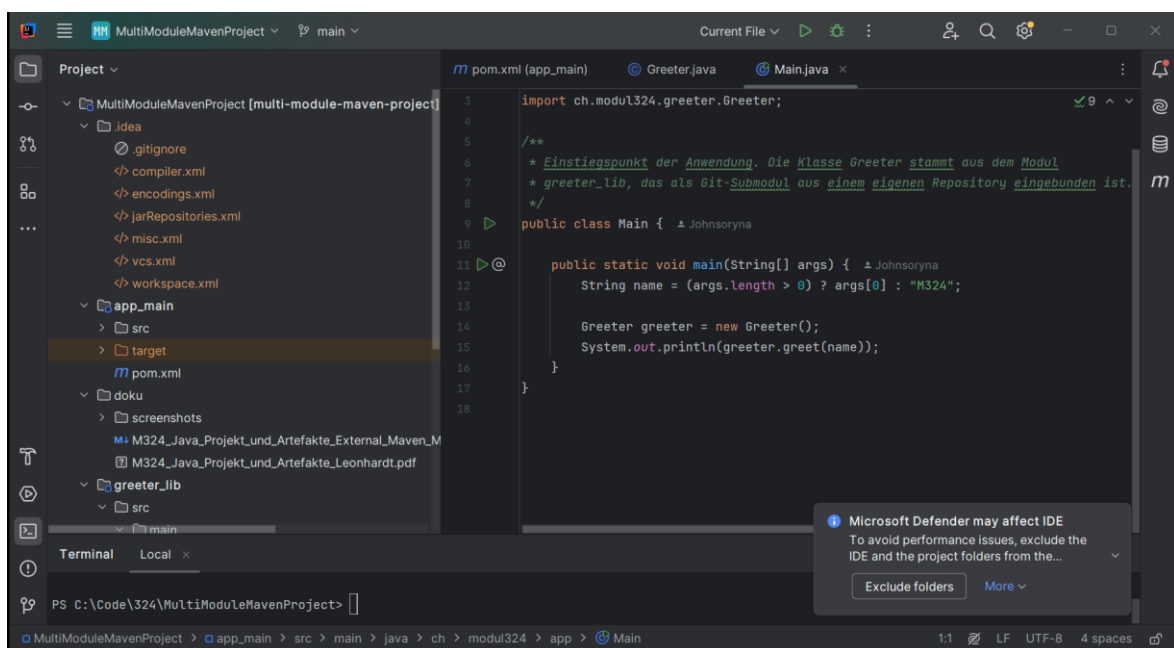
Lokales Verzeichnis des Hauptprojekts

Wichtige Stellen in den pom.xml:

```
<!-- Parent -->
<modules>
  <module>greeter_lib</module>
  <module>app_main</module>
</modules>

<!-- app_main: Abhängigkeit auf das andere Modul -->
<dependency>
  <groupId>ch.modul324</groupId>
  <artifactId>greeter_lib</artifactId>
</dependency>
```

Die Version steht im dependencyManagement des Parents, darum braucht app_main keine. Maven baut greeter_lib automatisch vor app_main, weil app_main davon abhängt.



```
Windows PowerShell
[INFO] Reactor Build Order:
[INFO] Greeter Library [jar]
[INFO] Multi Module Maven Project [pom]
[INFO] App Main [jar]
[INFO] Building Greeter Library 1.0.0 [1/3]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: C:\Code\324\MultiModuleMavenProject\greeter_lib\target\greeter_
lib-1.0.0.jar
[INFO] Building jar: C:\Code\324\MultiModuleMavenProject\greeter_lib\target\greeter_
lib-1.0.0-sources.jar
[INFO] Building Multi Module Maven Project 1.0-SNAPSHOT [2/3]
[INFO] Building App Main 1.0-SNAPSHOT [3/3]
[INFO] Building jar: C:\Code\324\MultiModuleMavenProject\app_main\target\app_main-1.
0-SNAPSHOT.jar
[INFO] Reactor Summary:
[INFO] Greeter Library 1.0.0 ..... SUCCESS [ 7.700 s]
[INFO] Multi Module Maven Project 1.0-SNAPSHOT ..... SUCCESS [ 0.027 s]
[INFO] App Main 1.0-SNAPSHOT ..... SUCCESS [ 2.138 s]
[INFO] BUILD SUCCESS
[INFO] Total time: 10.098 s

PS C:\Code\324\MultiModuleMavenProject> java -jar app_main\target\app_main-1.0-SNAPS
HOT.jar Johnny
Hallo, Johnny! Gruss aus greeter_lib 1.0.0.

PS C:\Code\324\MultiModuleMavenProject>
```

mvn clean install baut beide Module, danach Ausführung des JAR

greeter_lib als Git-Submodul

greeter_lib bekam ein eigenes Git-Repository, das Hauptprojekt (Parent + app_main) ein anderes. Beim ersten Commit im Hauptprojekt darf der Ordner greeter_lib nicht dabei sein, sonst schlägt git submodule add fehl.

```
cd greeter_lib
git init -b main
git add .
git commit -m "greeter_lib"
gh repo create m324-greeter-lib --public --source . --push

cd ..
git init -b main
git add .gitignore pom.xml app_main
git commit -m "Parent und app_main"
gh repo create m324-multimodule --public --source . --push
git submodule add https://github.com/Johnsoryna/m324-greeter-lib.git greeter_lib
git commit -m "greeter_lib als Submodul"
git push
```

Git legt die Datei .gitmodules an und speichert im Hauptprojekt nur einen Zeiger auf den Commit des Submoduls (Eintrag mit Modus 160000). Der Code selbst liegt nur im Repository der Bibliothek.

```
Windows PowerShell
PS C:\Code\324\MultiModuleMavenProject> cat .gitmodules
[submodule "greeter_lib"]
    path = greeter_lib
    url = https://github.com/Johnsoryna/m324-greeter-lib.git

PS C:\Code\324\MultiModuleMavenProject> git submodule status
3033c7a8a9bd441e415245dcf56946fa43f641c5 greeter_lib (heads/main)

PS C:\Code\324\MultiModuleMavenProject> git ls-files --stage greeter_lib pom.xml
160000 3033c7a8a9bd441e415245dcf56946fa43f641c5 0      greeter_lib
100644 417fbcdba09ad2b63a21127c53422645c50e1318 0      pom.xml
PS C:\Code\324\MultiModuleMavenProject> |
```

.gitmodules, Submodul-Status und der gitlink-Eintrag im Index

File	Commit Hash	Timestamp
app_main	Multi Module Maven Projekt: Parent-POM und Modul app_m...	2 weeks ago
doku	Doku: Abgabedokument fuer beide Aufgaben gekuerzt, Scree...	23 minutes ago
greeter_lib @ 3033c7a	greeter_lib-Submodul auf Release 1.0.0 aktualisiert	48 minutes ago
.gitignore	Multi Module Maven Projekt: Parent-POM und Modul app_m...	2 weeks ago
.gitmodules	greeter_lib als Git-Submodul aus eigenem Repository eingebu...	2 weeks ago
pom.xml	greeter_lib-Submodul auf Release 1.0.0 aktualisiert	48 minutes ago

Auf GitHub erscheint greeter_lib als Link mit Commit-Hash und fñhrt ins andere Repository

Klonen und Bauen habe ich in einem leeren Ordner getestet. Ein normaler git clone lässt greeter_lib leer und mvn clean install bricht ab (Child module greeter_lib/pom.xml does not exist). Mit git clone -recurse-submodules ist der Ordner gefüllt und der Build läuft durch.

Wenn sich das Submodul ändert, braucht es zwei Commits: zuerst im Submodul, dann im Hauptprojekt den neuen Zeiger (git add greeter_lib, commit, push). Das habe ich beim Wechsel auf Version 1.0.0 in der letzten Aufgabe gemacht.

Aufgabe: IDE-Ansatz vs. Maven/Gradle

Ohne Maven/Gradle in IntelliJ: JAR-Datei manuell herunterladen (z.B. von mvnrepository.com), dann File > Project Structure > Libraries > + > Java, JAR auswählen und dem Modul zuordnen. Ein eigenes Artefakt baut man unter Project Structure > Artifacts > + > JAR und dann Build > Build Artifacts. Die Konfiguration landet in .idea und .iml.

Mit Maven reicht ein Eintrag in der pom.xml, mit Gradle eine Zeile in build.gradle. Das Tool lädt das JAR samt transitiven Abhängigkeiten selbst und die IDE liest dieselbe Datei.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

```
implementation 'com.mysql:mysql-connector-j:8.4.0' // Gradle
```

	IDE-Ansatz	Maven / Gradle
Reproduzierbar	Nein, hängt an lokalen JARs und IDE-Dateien	Ja, Build-Datei ist im Repo
Transitive Abhängigkeiten	Selbst suchen	Automatisch
CI/CD	Nicht möglich, Build-Server hat keine IDE	mvn clean install läuft überall
Teamarbeit / IDE-Wechsel	Jeder muss JARs selbst besorgen, Lock-in	Jede IDE versteht Maven/Gradle
Einstieg	Einfach, nur klicken	Konzepte müssen gelernt werden

Für ein DevOps-Projekt kommt nur Maven oder Gradle in Frage. Der IDE-Weg ist höchstens zum schnellen Ausprobieren gut.

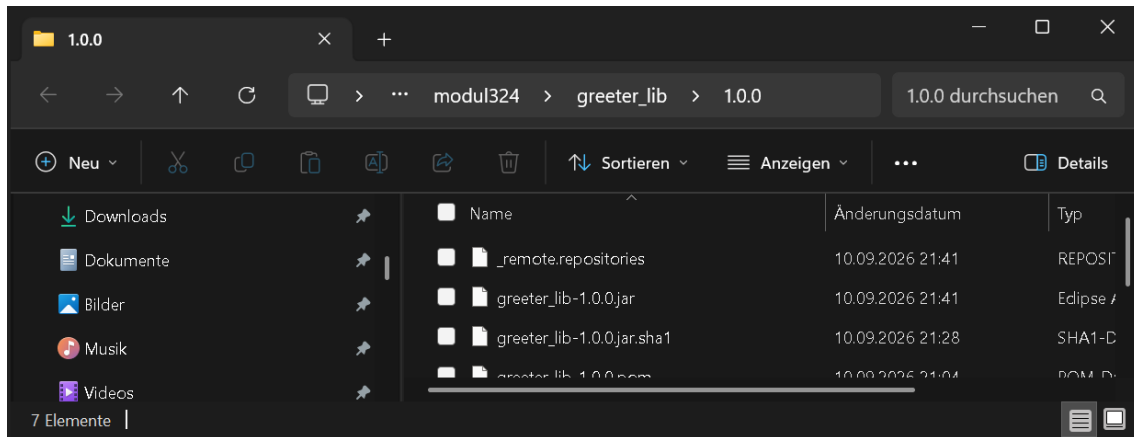
Aufgabe: Refresher: Was ist ein Maven-Artefakt?

Begriff	Bedeutung	Beispiel
groupId	Herausgeber, wie ein umgekehrter Domainname	ch.modul324
artifactId	Name des Artefakts	greeter_lib
version	Versionsnummer, optional mit -SNAPSHOT	1.0.0
packaging	Dateityp: jar (Standard), war, pom, ...	jar
classifier	Zusatz für weitere Dateien mit gleichen Koordinaten	sources, javadoc

groupId, artifactId und version (GAV) identifizieren ein Artefakt eindeutig. Aus den Koordinaten ergibt sich der Pfad im Repository:

```
<groupId als Ordner>/<artifactId>/<version>/<artifactId>-<version>[-
<classifier>].<packaging>
ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.jar
ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0-sources.jar
ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.pom
```

Zu jedem Artefakt gehört neben dem JAR immer ein .pom mit den Metadaten und den eigenen Abhängigkeiten. Nur damit kann Maven transitive Abhängigkeiten auflösen.



greeter_lib 1.0.0 im lokalen Repository

Release und Snapshot: Ein Release (1.0.0) ist unveränderlich, dieselbe Version hat immer denselben Inhalt. Ein Snapshot (1.0.0-SNAPSHOT) ist ein Entwicklungsstand, der bei jedem Deploy überschrieben wird. Maven prüft bei Snapshots täglich auf eine neue Version. Ein Release darf keine Snapshot-Abhängigkeiten haben, sonst ist der Build nicht reproduzierbar. Darum musste greeter_lib vor dem Publizieren auf 1.0.0 gestellt werden.

Aufgabe: Lokales Maven-Repository

Das lokale Repository ist ein Cache. Maven schaut bei jeder Abhängigkeit zuerst dort nach und lädt nur, was fehlt. mvn install legt auch die eigenen Artefakte dort ab, damit andere lokale Projekte sie nutzen können.

Pfad	C:\Users\john-.m2\repository
Grösse	347.8 MB
Dateien	7190

```
Get-ChildItem "$env:USERPROFILE\.m2\repository" -Recurse -File | Measure-Object -
Property Length -Sum
```

settings.xml: Sie enthält Einstellungen, die nicht ins Projekt gehören, vor allem Zugangsdaten. Es gibt sie global (conf/settings.xml der Maven-Installation) und pro Benutzer (~/.m2/settings.xml). Die pom.xml beschreibt das Projekt und wird versioniert, die settings.xml beschreibt den Rechner und kommt nie ins Git.

- servers: Benutzernamen und Passwort/Token pro Repository-id
- mirrors: Anfragen umleiten, z.B. auf einen Firmen-Nexus
- proxies: Proxy-Einstellungen
- localRepository: anderer Speicherort für das lokale Repository
- profiles / activeProfiles: zusätzliche Repositories und Properties

Auf meinem Rechner gab es vorher keine settings.xml. Für GitHub Packages habe ich diese angelegt, das Token kommt aus einer Umgebungsvariable:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0">
  <servers>
    <server>
      <id>github</id>
```

```

        <username>Johnsoryna</username>
        <password>${env.GITHUB_TOKEN}</password>
    </server>
</servers>
</settings>

```

Aufgabe: Artefakte zur Verfügung stellen

Publiziert wird immer mit mvn deploy. Das Ziel steht in der pom.xml unter distributionManagement, die Zugangsdaten in der settings.xml, verbunden über die gleiche id.

Möglichkeit	Vorgehen	Vorteile	Nachteile
Maven Central	Account im Central Portal, Namespace verifizieren (Domain oder io.github.<user>), GPG-Key, POM mit name/url/licenses/scm, Sources- und Javadoc-JAR, mvn deploy und freigeben	Weltweit ohne Konfiguration nutzbar, kostenlos	Aufwändig, nur Releases, nichts kann gelöscht werden, alles öffentlich
Nexus / Artifactory	Server betreiben oder hosten, Repos für releases/snapshots/proxy anlegen, Credentials in settings.xml, distributionManagement, mvn deploy	Privat, volle Kontrolle, Releases und Snapshots, Cache für Central	Betrieb und Kosten, für kleine Projekte zu viel
GitHub Packages	Token mit write:packages, server in settings.xml, distributionManagement auf maven.pkg.github.com/<owner>/<repo>, mvn deploy	Direkt beim Repo, keine eigene Infrastruktur, gratis für public	Auch zum Lesen braucht man ein Token, Bindung an GitHub
File-Sharing	Ordner mit Maven-Struktur auf Netzlaufwerk oder Webserver als file:// Repository, oder mvn install:install-file	Kein Server, sofort eingerichtet	Keine Rechte, keine Prüfsummen, manuelle Verteilung, nicht CI-tauglich

Aufgabe: External Maven Module (GitHub Packages)

Ziel: greeter_lib als Maven-Package auf GitHub publizieren und in einem anderen Projekt als normale Abhängigkeit verwenden, ohne Submodul und ohne Quellcode der Bibliothek.

Schritt 1 und 2: Repository und pom.xml

Verwendet habe ich das bestehende Repository m324-greeter-lib. In der pom.xml musste ich zwei Sachen ändern: die Version von 1.0-SNAPSHOT auf 1.0.0 (kein Snapshot erlaubt) und den parent-Block entfernen. Sonst würde das publizierte POM auf das Parent-Projekt zeigen, das in keinem Repository existiert, und der Konsument könnte es nicht auflösen. greeter_lib ist jetzt ein eigenständiges Artefakt und funktioniert im Multi-Module-Projekt trotzdem weiter.

```

<groupId>ch.modul324</groupId>
<artifactId>greeter_lib</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>

```

Schritt 3 und 4: Ziel und Zugangsdaten

```

<distributionManagement>

```

```

<repository>
  <id>github</id>
  <name>GitHub Packages</name>
  <url>https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib</url>
</repository>
</distributionManagement>

```

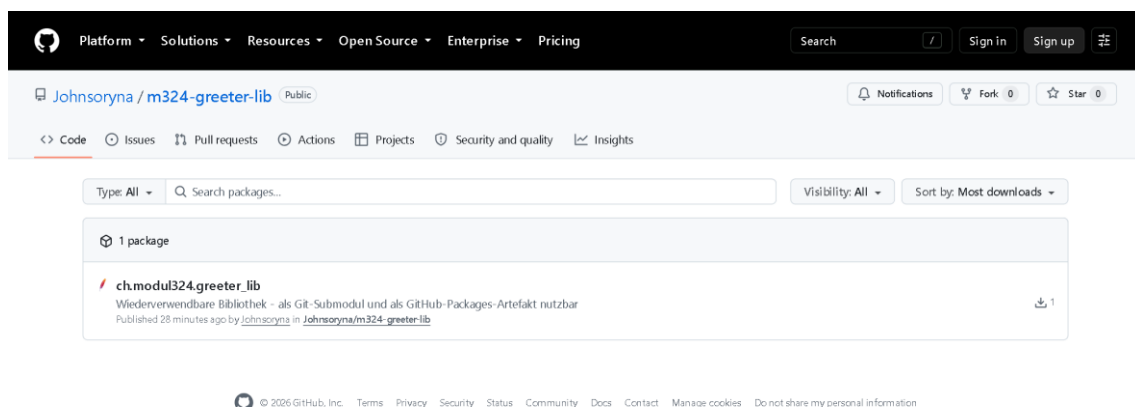
Das Token mit den Scopes write:packages und read:packages habe ich in GitHub erstellt und in der settings.xml (siehe oben) hinterlegt. mvn help:effective-settings zeigt den server-Eintrag ohne Fehler an.

Schritt 5: Publizieren

```

PS C:\Code\324\MultiModuleMavenProject\greeter_lib> mvn clean deploy
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
Uploading to github: https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib/ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.pom
Uploading to github: .../greeter_lib/1.0.0/greeter_lib-1.0.0.jar
Uploading to github: .../greeter_lib/1.0.0/greeter_lib-1.0.0-sources.jar
Uploading to github: .../greeter_lib/maven-metadata.xml
[INFO] BUILD SUCCESS

```



Das Package im Repository unter Packages

Schritt 6: In einem anderen Projekt verwenden

Neues Projekt GreeterConsumer mit nur einer pom.xml und einer Klasse. Weil Maven standardmässig nur Maven Central kennt, muss das GitHub-Repository eingetragen werden:

```

<repositories>
  <repository>
    <id>github</id>
    <url>https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib</url>
  </repository>
</repositories>

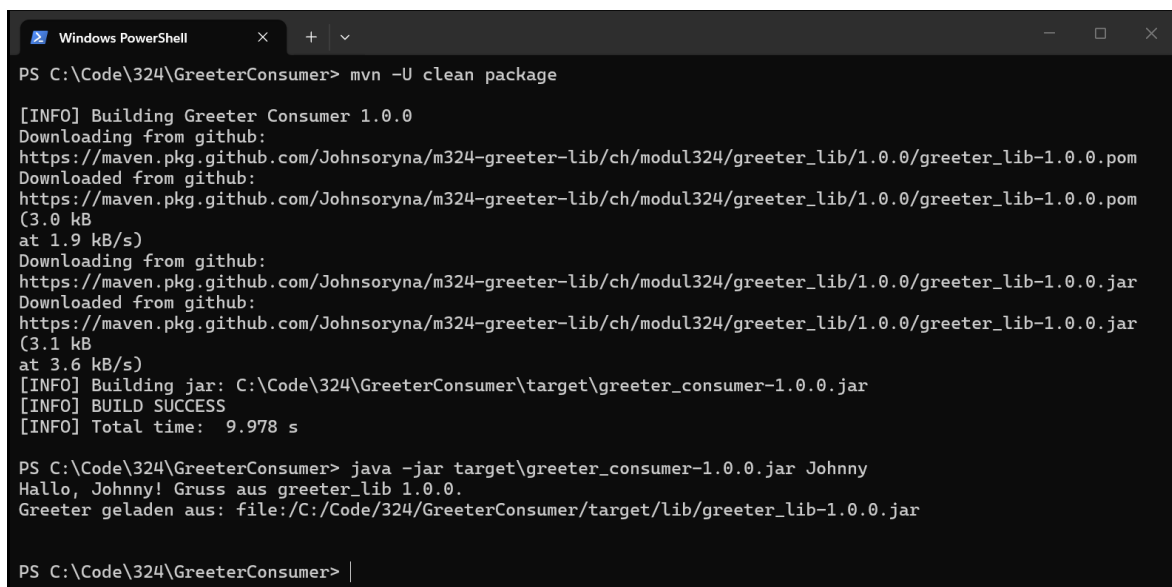
<dependency>
  <groupId>ch.modul324</groupId>
  <artifactId>greeter_lib</artifactId>
  <version>1.0.0</version>
</dependency>

```

```
import ch.modul324.greeter.Greeter;

public class ConsumerApp {
    public static void main(String[] args) {
        String name = (args.length > 0) ? args[0] : "M324";
        System.out.println(new Greeter().greet(name));
        System.out.println("Greeter geladen aus: "
            + Greeter.class.getProtectionDomain().getCodeSource().getLocation());
    }
}
```

Damit der Download von GitHub wirklich getestet wird, habe ich greeter_lib vorher aus dem lokalen Repository (.m2) gelöscht und mit -U gebaut. Maven lädt das POM und das JAR von maven.pkg.github.com, der Build ist erfolgreich und das Programm gibt die Begrüssung aus dem externen Modul aus.



```
Windows PowerShell
PS C:\Code\324\GreeterConsumer> mvn -U clean package

[INFO] Building Greeter Consumer 1.0.0
Downloading from github:
https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib/ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.pom
Downloaded from github:
https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib/ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.pom
(3.0 kB
at 1.9 kB/s)
Downloading from github:
https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib/ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.jar
Downloaded from github:
https://maven.pkg.github.com/Johnsoryna/m324-greeter-lib/ch/modul324/greeter_lib/1.0.0/greeter_lib-1.0.0.jar
(3.1 kB
at 3.6 kB/s)
[INFO] Building jar: C:\Code\324\GreeterConsumer\target\greeter_consumer-1.0.0.jar
[INFO] BUILD SUCCESS
[INFO] Total time: 9.978 s

PS C:\Code\324\GreeterConsumer> java -jar target\greeter_consumer-1.0.0.jar Johnny
Hallo, Johnny! Gruss aus greeter_lib 1.0.0.
Greeter geladen aus: file:/C:/Code/324/GreeterConsumer/target/lib/greeter_lib-1.0.0.jar

PS C:\Code\324\GreeterConsumer> |
```

Download von GitHub Packages und Ausführung des Konsumenten

Ohne Token (GITHUB_TOKEN leer) bricht der Build mit 401 Unauthorized ab. GitHub Packages verlangt also auch für öffentliche Pakete eine Anmeldung. Das ist der grösste Nachteil gegenüber Maven Central.

Quellen

- <https://www.baeldung.com/maven-artifact>
- <https://www.baeldung.com/maven-local-repository>
- <https://git-scm.com/book/en/v2/Git-Tools-Submodules>
- <https://www.jetbrains.com/help/idea/artifacts.html>
- <https://maven.apache.org/settings.html>
- <https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-apache-maven-registry>